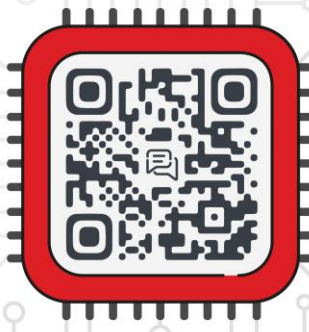




5



6



900 sp

عملي

كلية الهندسة المعلوماتية

السنة الثانية

B_tree

عملي مشترك



RB Informatics; 16/5/2026

محتوى مجاني غير مخصص للبيع التجاري

خوارزميات ٢

بسم الله الرحمن الرحيم

B_tree

العناوين الرئيسية:

تعريف B_tree وخصائصها

العمليات الأساسية

تمارين

تنبيه يرجى دراسة محاضرة النظري الخاصة بال B tree قبل دراسة هذه المحاضرة 😊🙏
حيث سنقوم في هذه المحاضرة بمراجعة الأفكار الأساسية والتركيز على ترجمة هذه الأفكار لأكواد بلغة جافا

تعريف B_tree

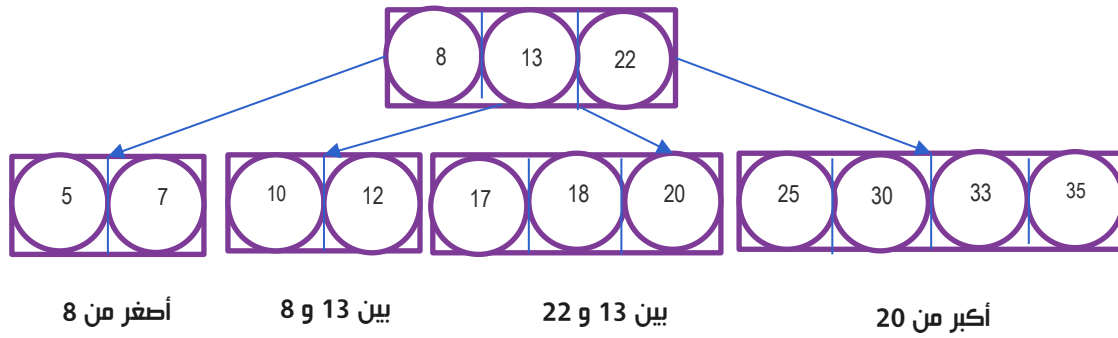
هي شجرة بحث متوازنة تخزن فيها البيانات على شكل صفحات pages وتحتوي كل صفحة على عدة مفاتيح مرتبة حيث يمكن تمثيل الصفحة بمصفوفة أو لينكد ليست

خصائصها:

- مفاتيح كل صفحة مرتبة تصاعدياً أو تنازلياً
- كل صفحة تحتوي من t إلى $2t$ مفاتيح ، عدا الجذر حيث يمكن أن يحتوي من 1 إلى $2t$ مفاتيح حيث t درجة الشجرة

جميع الأوراق في نفس المستوى
إذا كان للصفحة m مفتاح:
فإنها أن لا يكون لها أبناء أو يكون لها $m+1$ ابن

مثال: لدينا B_tree جانباً حيث $t=2$:



العمليات الأساسية:

البحث search :

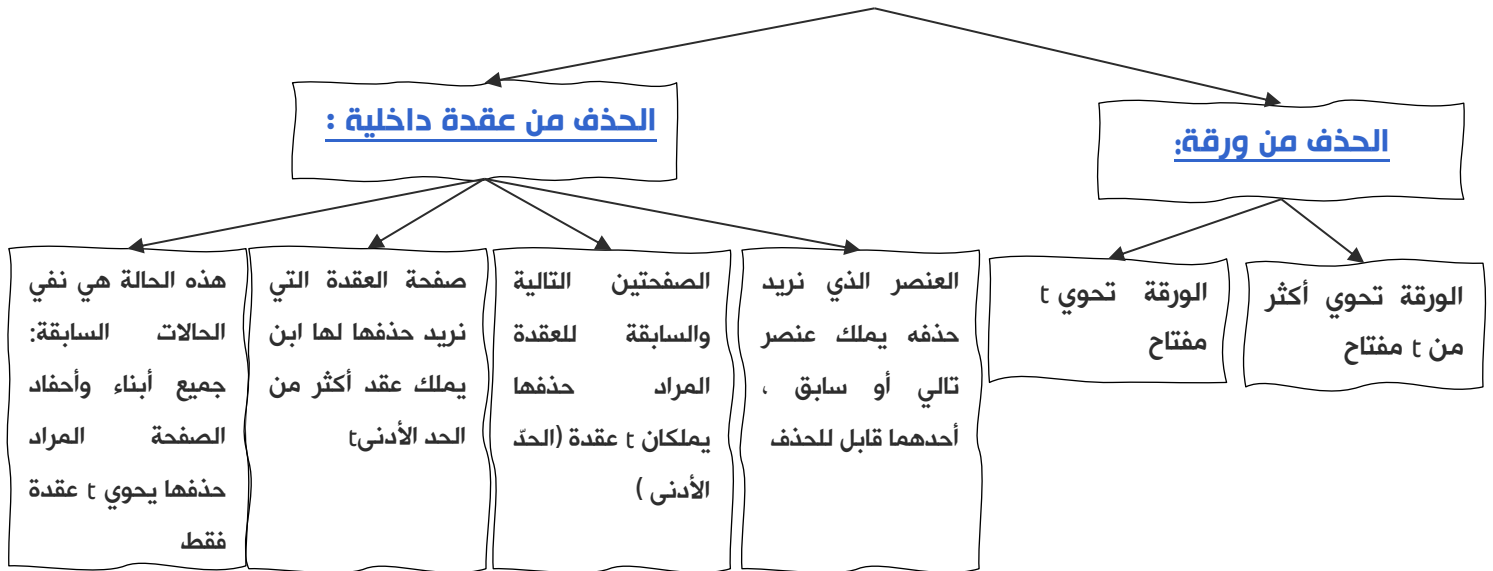
ويبدأ من الجذر
داخل كل صفحة يتم البحث عن المفتاح المطلوب (باستخدام البحث الخطي أو الثنائي)
إذا وُجد المفتاح تُرجع مكانه
وإلا ينتقل إلى الابن المناسب (بناءً على مقارنة المفاتيح) ونكرّر العملية حتى الوصول إلى ورقة أو إلى العثور على المفتاح المطلوب ☺

الإدراج insertion :

الإضافة هنا تكون حصراً في الأوراق
يتم البحث عن الورقة المناسبة لإضافة المفتاح الجديد
إذا كانت الورقة غير ممتلئة (عدد مفاتيحها أقل من $2t$):
ندرج المفتاح بالترتيب
أو إذا كانت ممتلئة (عدد مفاتيحها يساوي $2t$):
فعندئذ نقسم الورقة إلى ورقتين كل منهما تحوي على t مفتاح والمفتاح الأوسط يُرفع إلى الأب
إذا كان الأب ممتلئاً أيضاً نكرّر عملية التقسيم صعوداً إلى الجذر

سنذكر الحالات فقط بشكل عام، الشرح الكامل بمحاضرة النظري (٩٠٠)

نميز حالتين للحذف:



تمارين:

قُم بكتابة برنامج بلغة Java ينشئ شجرة B-Tree تحتوي على مجموعة من الأعداد الصحيحة، ويجب أن يتضمن البرنامج التوابع التالية:

- تابع لإدراج الأعداد في الشجرة
- تابع لتقسيم العقدة إذا كانت ممثلة
- تابع للإدراج العدد في العقدة المناسبة
- تابع لطباعة الشجرة
- تابع للبحث في الشجرة

الخوارزميات أرضي وملعب
طبعاً أنا الطابة 🤔🤔



إدراج الأعداد في الشجرة

```
private static class BTreeNode {
    int[] keys;
    int t;
    BTreeNode[] children;
    int numKeys;
    boolean leaf;

    public BTreeNode(int t, boolean leaf) {
        this.t = t;
        this.leaf = leaf;
        keys = new int[2 * t - 1];
        children = new BTreeNode[2 * t];
    }
}

private BTreeNode root;
private int t;

public BTreeExample(int t) {
    this.t = t;
    root = new BTreeNode(t, true);
}

public void insert(int key) {
    BTreeNode r = root;
    if (r.numKeys == 2 * t - 1) {
        BTreeNode s = new BTreeNode(t, false);
        root = s;
        s.children[0] = r;
        splitChild(s, 0);
        insertNonFull(s, key);
    } else {
        insertNonFull(r, key);
    }
}
```

تقسيم العقدة إذا كانت ممتلئة

```
private void splitChild(BTreeNode x, int i) {
    BTreeNode y = x.children[i];
    BTreeNode z = new BTreeNode(t, y.leaf);
    x.numKeys++;

    for (int j = t - 1; j >= 0; j--) {
        z.keys[j] = y.keys[j + t];
        y.keys[j + t] = 0;
    }

    if (!y.leaf) {
        for (int j = t; j >= 0; j--) {
            z.children[j] = y.children[j + t];
            y.children[j + t] = null;
        }
    }
}
```



```

for (int j = x.numKeys - 1; j > i; j--) {
    x.children[j + 1] = x.children[j];
}

x.children[i + 1] = z;

for (int j = x.numKeys - 2; j >= i; j--) {
    x.keys[j + 1] = x.keys[j];
}

x.keys[i] = y.keys[t - 1];
y.keys[t - 1] = 0;
y.numKeys = t - 1;
}

```

طباعة شجرة

```

private static void printBTree(BTreeNode node) {
    for (int i = 0; i < node.numKeys; i++) {
        System.out.print(node.keys[i] + " ");
    }
    System.out.println();
    if (!node.leaf) {
        for (int i = 0; i <= node.numKeys; i++) {
            printBTree(node.children[i]);
        }
    }
}

```

إدراج العدد في العقدة المناسبة

```

private static class BTreeNode {
    int[] keys;
    int t;
    BTreeNode[] children;
    int numKeys;
    boolean leaf;

    public BTreeNode(int t, boolean leaf) {
        this.t = t;
        this.leaf = leaf;
        keys = new int[2 * t - 1];
        children = new BTreeNode[2 * t];
    }
}

private BTreeNode root;
private int t;

public BTreeExample(int t) {
    this.t = t;
    root = new BTreeNode(t, true);
}

public void insert(int key) {
    BTreeNode r = root;

```



```

if (r.numKeys == 2 * t - 1) {
    BTreeNode s = new BTreeNode(t, false);
    root = s;
    s.children[0] = r;
    splitChild(s, 0);
    insertNonFull(s, key);
} else {
    insertNonFull(r, key);
}
}

```

البحث في الشجرة

```

private boolean search(BTreeNode node, int key) {
    int i = 0;
    while (i < node.keyCount && key > node.keys[i]) {
        i++;
    }
    if (i < node.keyCount && node.keys[i] == key) {
        return true;
    }
    if (node.isLeaf) {
        return false;
    }
    return search(node.children[i], key);
}

```



المحاضرة كانت لطيفة وظريفة ☺
 نلتقاكم بمحاضرات قادمة
 بالتوفيق والسداد (٧)

